

FUNCTIONALITY REPORT

Project Claude Blueprint

How it works, and what a project inherits from it

A guided tour for human readers

Blueprint version 1.2.0 · commit `b2df960` (2026-08-14)

Report generated 2026-08-25 · compiled from source by `CLAUDE_blueprint_report`

7 decisions · 7 CI gates · 7 harness hooks · 11 ritual commands

Contents

1	What the blueprint is	3
	The problem it solves, the shape of the answer, and whether it is for you.	
2	The documentation spine	7
	Why the docs site is canonical, what the template fills, and the space it leaves your project.	
3	Decisions that hold still	11
	The ADR registry, and the lock that stops a decision being quietly rewritten.	
4	Enforcement: rules with teeth	13
	Layered gates, declared failure directions, and exceptions that leave a record.	
5	The agent harness	18
	What a session is told, permitted, and prevented from doing — and why the telling stays small.	
6	The lifecycle rituals	23
	Birth, staying current, retrofitting, and feeding improvements back.	
7	Bootstrap choices	26
	The interview, the binary posture, and what is deferred to you afterwards.	
8	What a project inherits	29
	Concretely, what exists in a repository the moment bootstrap finishes.	
9	Colophon	31
	How this report is produced and kept honest.	

What the blueprint is

Project Claude Blueprint is a GitHub template repository. You create a new repository from it, point an AI coding session at the result, and one bootstrap session later you have a project that already knows how it is supposed to be built — before a single line of its actual subject matter exists.

What it installs is not code. It is **process**: a documentation site that is authoritative rather than decorative, a registry of decisions that cannot be quietly rewritten, and a set of gates that enforce both. The wager is that these are the things a project cannot retrofit cheaply, and therefore the things worth having on day one.

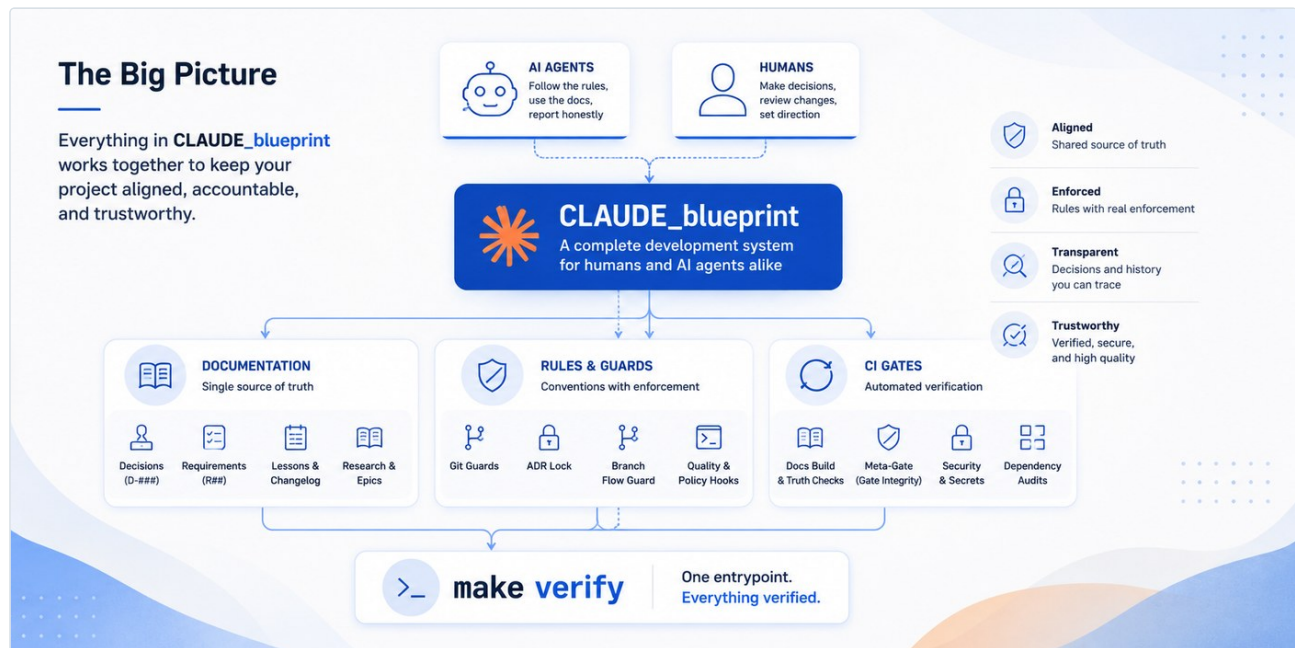
7 DECISION RECORDS	7 CI GATES	7 HARNESS HOOKS	16 PROCESS PAGES	22 REPO SCRIPTS
---------------------------------	----------------------	---------------------------	----------------------------	---------------------------

The problem it addresses

A project's conventions are usually discovered rather than decided. Someone picks a branch name, someone else copies it, and eighteen months later the shape of the repository is an accumulation of accidents that nobody chose and nobody can now afford to change. Documentation drifts from the code because nothing checks it. Architectural decisions live in chat logs and in people's heads, so the reasoning is lost precisely when someone proposes reversing them.

This gets sharper when the main contributor is an AI agent. An agent is fast, literal, and has no memory between sessions. It will follow whatever conventions it can find, and invent plausible ones where it finds none. The cost of an unwritten rule rises steeply: a human colleague absorbs the local style by osmosis, whereas an agent re-derives it from scratch every session and gets it subtly wrong in a different way each time.

The tempting fix — write every rule into the one file the agent always reads — collapses under its own success. A context window is finite, and standing instructions compete with the actual work for it; a guidance file that grows with the project ends up crowding out the project. So the blueprint's wager has a second half: structure beats bulk. The always-loaded file stays a small map, and every rule it points to loads only at the moment a task makes it relevant — which, the bet goes, outperforms a long rulebook that tries to hold everything at once. How that economy is engineered is the harness chapter's subject.



How the pieces fit together: documentation, guards and CI gates, serving both human and agent contributors behind a single verify command

The shape of the answer

The blueprint's response has five moving parts, and the rest of this report is an examination of each in turn.

A canonical documentation site. Everything true about the project lives in `docs/`, built with MkDocs Material, and the build runs in strict mode as a merge gate — a broken cross-reference fails CI. Every fact has exactly one home, so there is no second copy to fall out of date.

An addressable decision registry. Significant choices become Architecture Decision Records with stable identifiers (`D-001`, `D-002`, ...). A decision marked Decided is immutable: changing your mind means writing a new record that supersedes the old one, and the old reasoning stays readable.

Layered enforcement. Each rule names the thing that enforces it, or is explicitly labelled advisory with a reason. Enforcement runs in layers — prose, then client-side hooks, then the server: CI gates with branch protection behind them — and each layer declares which way it fails when it breaks.

An agent harness. Everything in `.claude/` that turns the written process into what an AI session actually does — permissions, guards, rituals, skills — engineered so the guidance a session always carries stays small.

A lifecycle. Four rituals cover a project's whole relationship with the template: birth, staying current as the template improves, retrofitting the process into a project that was never seeded from it, and harvesting improvements back out of live projects into the template.

When the blueprint fits

The blueprint is opinionated on purpose, which means it is also opinionated about who it is for. Before anything else in this report, here is the honest checklist. Each item is an assumption the design leans on; tick it if it is true of your project. The more you can tick, the more the machinery works for you — the more you leave unticked, the more you will find yourself fighting guards built for someone else's situation.

- ☑ **Your project is long-lived and will accumulate.** The machinery earns its keep by stopping the slow failures — drifting conventions, re-litigated decisions, docs that quietly stop being true — and those compound with size and age. On a project that will grow code and documentation for years, the setup cost disappears into the noise. On a weekend prototype it is the noise.
- ☑ **You are on GitHub.** This one is close to hard. The binding enforcement layer is GitHub Actions plus GitHub branch protection; work tracking uses native sub-issues and labels; setup drives the GitHub API. On another forge the docs and the ideas transfer, but every server-side gate — the layer the whole doctrine says is the authority — would have to be rebuilt.
- ☑ **Work arrives as issues and leaves as pull requests.** The flow is not decoration: branches merge only through PRs, PRs close issues by gated keyword, and issue closing itself is guarded. A team that trunk-commits to main, or tracks work anywhere but the issue tracker, would be stepping around the process at every turn.
- ☑ **AI agents do a real share of the building — Claude Code first among them.** The premise of the harness chapter. Any agent, and any human, is bound by the same rules through CI, and an `AGENTS.md` routes other tools to the one rulebook — but the edit-time guards, the ritual commands, the skills, and the permission lists are Claude Code formats and fire only there. Other agents follow the process by hand; a purely human team keeps the decision and docs discipline but carries harness machinery it never uses.
- ☑ **Documentation lives in MkDocs Material.** The strict build is a merge gate and the truth checker reads the site's nav, so the docs system is load-bearing, not a suggestion. A different generator — or a wiki — means rebuilding the documentation gates around it.
- ☑ **A human operator stays in the loop.** Someone merges pull requests, closes issues, decides version numbers, and mints the unlock token when a settled decision genuinely needs maintenance. This is not a review bureaucracy — branch protection requires passing checks, not approvals, so a single operator working with agents is the designed case — but fully hands-off autonomy is not on offer.
- ☑ **You will take the process as shipped.** Bootstrap seeds a complete, working process, and a seeded project extends it — its own decisions from `D-008`, its own domain areas — rather than reshaping it. If what you want is to keep your own branch model and records system and borrow only the ideas, that is not a tick you can make here.

An existing project that cannot tick the boxes wholesale is not turned away — it is pointed at a different door. The retrofit ritual in the lifecycle chapter adopts the invariants onto whatever shape a mature project already has, in operator-approved waves, without moving anything to match the template. The checklist above is for the clean-slate case, where the whole point is taking the shape too.

THE TEMPLATE IS ITS OWN FIRST CUSTOMER

The blueprint repository runs the rules it ships. Its own decisions are recorded as ADRs under the lock its own hook enforces; its own pull requests pass the gates it installs; `make verify` runs here exactly as it will downstream. The few places where being a template forces a difference — a records rule that only makes sense downstream — are declared exceptions in its guidance file, never quiet ones. A rule that proved unworkable in practice was changed in the template rather than tolerated in the projects — which is why the guard hooks carry regression suites, and why several of the sharper rules exist at all.

The documentation spine

The documentation site is not a description of the project. It **is** the project's specification, and the code is judged against it. This inverts the usual relationship, in which docs are written after the fact and rot quietly thereafter.

One fact, one home

The governing rule is canonicity: every fact has exactly one canonical location, and everything else links to it rather than restating it. Duplication is treated as a defect, because two copies of a fact are really one fact and one future lie. The doctrine page itself states it:

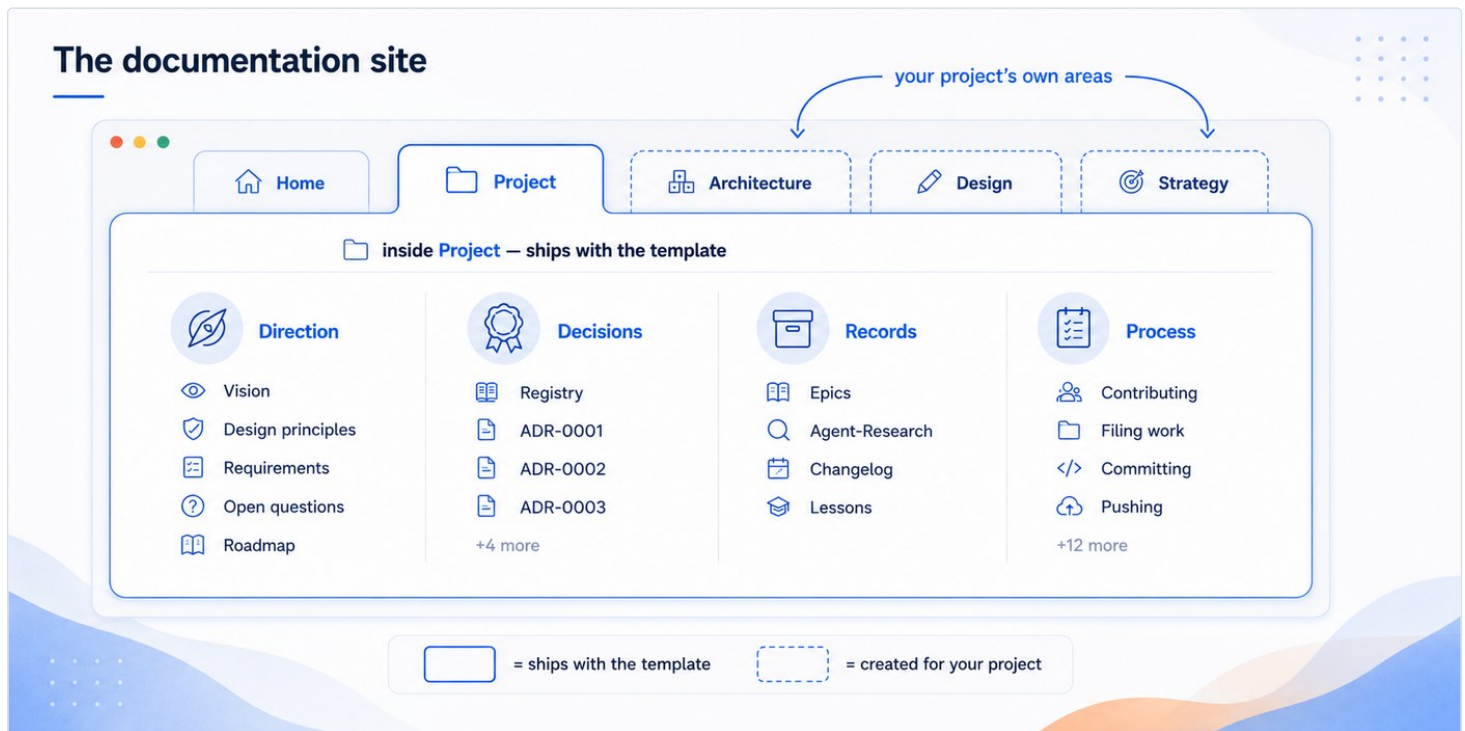
The docs site is canonical — it supersedes any working notes or playbooks.

- The decisions registry is the **authoritative list** of every `D-xxx` decision with its one-line statement + status.
- **Topic pages** hold the **detailed analysis** behind each decision. Each fact has exactly one canonical home; ADRs link to the topic pages rather than duplicating them. (Advisory — no gate detects duplicated analysis; the single-home discipline is upheld by review, per D-004.)
- Decision IDs (`D-xxx`), requirement IDs (`R##`), question IDs (`Q##`), and principle IDs (`P#`) are **shared across all pages** — there is one `D-004` , referenced from wherever it is relevant.
- When a decision's status changes, update its registry row and its ADR together so they stay in sync (gated by `registry-sync` — Writing ADRs).

This is why the blueprint's own agent-guidance file is deliberately thin. It is a map into `docs/` , not a summary of it — a summary would be a second copy, and the second copy always drifts.

How the site is organised

The site has a deliberately shallow top level, and the shape of it is the thing most descriptions get wrong. The sections this report has been discussing are not the site — they are what sits **inside one tab of it**.



Home and Project ship with the template; the domain-area tabs beside them are each project's own, and the template ships none

Home is the landing page. **Project** holds the four meta sections the template ships, identical in every seeded repository. Everything beside them is the project's own.

The four inside Project each have a distinct job.

Direction holds intent: the vision, the design principles (P1 , P2 , ...), the requirements every technical choice is judged against (R01 , R02 , ...), the questions still open (Q01 , Q02 , ...), and the roadmap. They are ordered as a traceability chain — vision informs principles, principles constrain requirements, requirements justify decisions.

Decisions holds the ADRs and the registry that indexes them: the authoritative record of what was chosen and why.

Records holds history — the session changelog, lessons learned, epic retrospectives, dated agent-research reports. Records narrate; they never decide. A finding that ought to change direction is promoted into a decision or an open question, and the report remains as the account of how the direction was found.

Process holds the act pages: `adding-docs-pages`, `closing-issues`, `committing`, `contributing`, `ending-a-session`, `enforcement`, `filing-work`, `glossary`, `opening-a-pr`, `pushing`, `records-and-canon`, `release-checklist`, `releases`, `running-epics`, `testing-changes`, `writing-adrs`. Most answer "how do I do this specific thing here" — open a pull request, close an issue, write an ADR, cut a release; the rest are the doctrine pages and the glossary the act pages lean on.

The space the template leaves you

Those four are meta: they are about running the project. None of them is where the project's actual subject matter goes.

That space is the **domain-area tabs**, and they sit beside Project rather than inside it. A project names its first areas at bootstrap — architecture, design, strategy, hardware, whatever the work is actually about — and adds more as they emerge over its life; each becomes its own top-level tab backed by its own directory under `docs/`. The blueprint ships none of them, which is why the diagram above draws them dashed. They are the one place the template is deliberately empty-handed, because it has nothing useful to say about a subject it does not know.

The separation earns its keep in the cross-references. An ADR under Decisions records that a choice was made and why; the topic page under a domain area explains how the thing actually works. The decision links to the topic page rather than restating it, so neither drifts.

This is also the one sanctioned exception to the shallow top level. A domain-area tab, whenever it appears, needs no decision record. Promoting any other section to a top-level tab is a structural change that gets an ADR and a registry row like anything else — advisory, since no gate detects a new non-area tab, and caught at review.

Stable identifiers

Identifiers are load-bearing. `D-004` means one thing forever; it is never renumbered, never reused, and a new item takes the next free number even if earlier ones were withdrawn. The same discipline applies to requirements, open questions, and principles.

The point is addressability. A commit message, a pull request, a code comment, or a conversation can cite `D-004` and the reference stays valid for the life of the project. Renumbering would silently invalidate every citation ever written, including the ones in git history that nobody can go back and fix.

A single status legend runs across all of them:

BADGE	MEANING
DECIDED	Settled, or done
PROPOSED	Under consideration, or in progress
OPEN	Unanswered, awaiting a decision
SUPERSEDED	Deferred, or replaced by a later record

(The docs site renders these as coloured emoji; this report substitutes text badges, which stay legible at print sizes.)

The build is a gate

`mkdocs build --strict` runs on every pull request as part of `make verify`. Strict mode promotes warnings to errors, so a link to a page that was renamed or deleted fails the build rather than producing a quietly broken site — and so does the reverse, a page sitting under `docs/` that no nav entry lists, so nothing publishable can be unreachable and nothing unreachable can hide.

A second checker goes further than link validity and tests whether the docs are still true: it fails on citations to paths that no longer exist, on issues cited as open that have since closed, on references to command-line flags and environment variables that have been removed, on status drift between an ADR and its registry row, on duplicate decision identifiers, on epic pages describing work as in progress when the tracking issue is closed, and on the mirror image — a sub-issue an epic page lists as built that is in fact still open.

That last category is the interesting one. It catches the specific failure where documentation is syntactically perfect and semantically stale — the kind a link checker cannot see, and the kind that misleads every reader until someone notices.

There is also a context economy riding on these gates. The thin map of pointers saves anything only while a machine checks that every pointer resolves and every page stays true — an agent that must re-verify what it reads has lost the discount. Verification is what makes the map affordable.

Decisions that hold still

Architecture Decision Records are not new. What the blueprint adds is enforcement: a record marked **DECIDED** is mechanically protected against edits, so the historical reasoning cannot be tidied away by someone who has since changed their mind.

The registry

The registry below is read directly out of the blueprint at build time. These are the decisions the template itself ships with — the process invariants a seeded project inherits and can then extend with its own.

ID	DECISION	STATUS
D-001	Documentation & decision records — the docs site is the single source of truth (strict build as a merge gate); stable typed IDs with one status legend; decisions live as ADRs + registry rows, superseded never rewritten ...	DECIDED
D-002	Work tracking — issues track work while docs hold truth — epics group native sub-issues (build tasks); findings are <code>note</code> -labelled issues, never sub-issues; notes are triaged at epic closeout ...	DECIDED
D-003	Branch model — feature branch → PR into <code>development</code> (the integration branch and default) → promotion PR into <code>main</code> ...	DECIDED
D-004	Enforcement doctrine — every process rule names its enforcer or is explicitly advisory with a reason ...	DECIDED
D-005	Testing policy — scaffolding ships binding hermetic self-tests inside <code>make verify</code> ; test-writing is event-driven (bugfix → regression test named on the <code>Pin:</code> line), placement follows the project's layout ...	DECIDED
D-006	Verification & honesty — <code>make verify</code> plus exercising the change is the bar for "done"; reporting is honest — a ticked box was actually run, numbers name source and <code>n=</code> , capability claims state where they exist ...	DECIDED
D-007	Binary hygiene — posture finalized at bootstrap — default strict/split (no binaries in source history; durable artifacts live outside it ...	PROPOSED

Each row summarises a full record on the docs site, where the context, the consequences, the alternatives considered, and an honest statement of how reversible the choice is all live.

One row is not like the others. D-007 ships **PROPOSED** on purpose: the binary posture is the one choice only a real project can finish, and the bootstrap-choices chapter shows how it is finalised.

Why immutability matters

The temptation with a decision record is to keep it current — to go back and edit D-003 when the branch model changes. The blueprint forbids this, and the reason is that an ADR is not a description of the present. It is a record of a judgement made at a particular time with particular information.

Editing it destroys the only evidence of why the original choice looked right, which is exactly the evidence you need when someone proposes reverting to it two years later. A superseded decision that remains readable is a warning sign preserved; a rewritten one is a trap reset.

So a changed decision becomes a **new** record with a new identifier, which supersedes the old. The old one flips to **SUPERSEDED** and stays where it is. Deferral gets the same discipline from the other side: a deferred item always names the trigger that will reactivate it, so nothing is parked open-endedly.

The lock

Because "don't edit Decided records" is exactly the kind of rule an agent will breach with the best of intentions while tidying something adjacent, it is enforced rather than requested.

A client-side hook denies every path to changing a **DECIDED** ADR page — edits, deletion, renaming, even creating a new page already stamped Decided. Getting past it takes a deliberate, auditable act: the operator issues an unlock token scoped to that specific record, valid for one hour. Legitimate maintenance — fixing a typo, repairing a link the docs truth checker flagged — remains possible; it just leaves a trace.

CI backs the hook up, because a client-side hook only protects sessions that actually run it. The server-side gate checks that a pull request touching a Decided record carries the corresponding trailer in one of its commits, so a contributor with hooks disabled cannot slip an edit through.

TWO LAYERS, DIFFERENT FAILURE MODES

The hook and the CI gate are not redundant. The hook gives immediate feedback in the session, where the fix is cheap, but it fails open — a broken or bypassed hook lets the write through. The CI gate fails closed and catches what the hook missed, but only once a pull request exists. Each covers the other's weakness, and the blueprint's enforcement doctrine requires both to state which way they fail.

What counts as a decision

Not everything needs a record. The blueprint's stated bar is simply a significant decision — and what the template then demands of the record shows what that means in practice: a mandatory statement of how reversible the choice is, and the alternatives considered, each with the one honest reason it lost. The record exists for the future contributor who proposes reversing it.

Choices within an already-decided boundary do not need their own record; they are implementation, and an agent is expected to make them and move on. The distinction is deliberately a matter of judgement, and it is one of the places the blueprint accepts advisory guidance over a mechanical rule.

Enforcement: rules with teeth

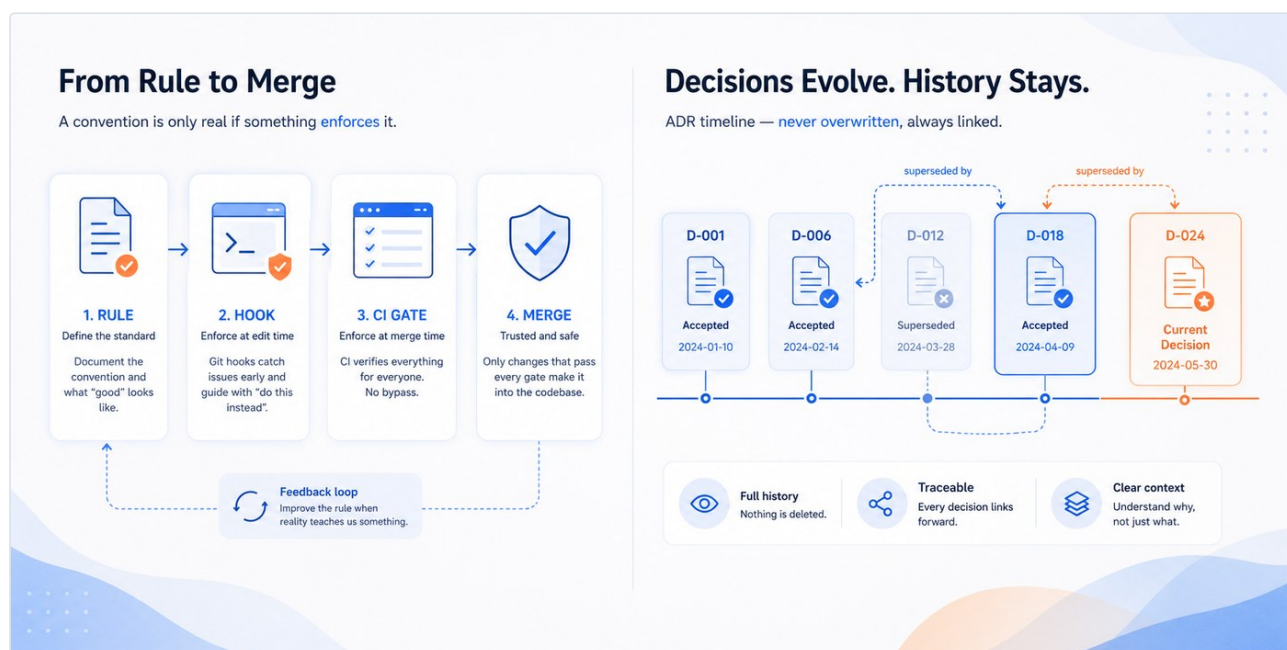
A convention nobody checks is a suggestion. The blueprint's fourth decision, `D-004`, is the one that makes the rest of the system hold. Its own summary — extracted from the record at build time — packs the whole doctrine into one block, and the rest of this chapter unpacks it:

Every process rule names what enforces it — or is explicitly advisory; enforcement is layered and every layer declares which way it fails; exceptions are reasoned and ledgered.

- **Rules declare their enforcement.** A rule added to `CLAUDE.md` or the process docs states the hook, CI gate, test, or template section that catches violations — or carries the honest label "advisory — deliberately unenforced because (reason)". The reference is bidirectional: the enforcing artifact cites the rule or decision it enforces in a comment, so config files explain themselves.
- **Enforcement is layered**, from cheap to binding: prose states the rule and points at its enforcer → **local hooks** nudge or deny at edit time, every block message naming the recovery path → **CI gates** are the binding layer that survives every client configuration. Hooks are code: every deny-hook ships a regression suite asserting both the deny side and the still-allowed side. A meta-gate pins the gate wiring itself, so a gate cannot be silently unwired.
- **Every layer declares its failure direction.** Deny-hooks block proven intent and fall through when their own machinery breaks — a broken guard must never brick every tool call — but an ambiguous **resolved** target blocks: uncertain evidence over-blocks safely; broken machinery falls through. Approve-hooks approve only what they can prove safe, defer everything else, and never block — a gap costs a prompt, never a mistake. The permission-prompt layer counts for nothing here (it is mode-dependent). CI gates and branch protection fail closed and are the authority.
- **Deferring a bypass class needs a named catcher.** A client guard may declare an evasion class out of scope (nested interpreters, command wrappers, time-of-check races) only by naming the server layer that catches it — and that claim is auditable (D-006), never assumed. A rule no server layer can carry is honestly labeled advisory at that boundary.
- **Server rules bind administrators.** The shipped branch-protection default is `enforce_admins: true` on the long-lived branches. The operator's escape hatch survives as a temporary, visible settings edit — relax, act, restore: the settings-layer analogue of a `Skip-` trailer — never an ambient exemption, because agents inherit the operator's identity and, with it, any standing bypass.
- **Sanctioned exceptions are commit trailers** with a mandatory reason — the generic `Skip-<Rule>: <one-line reason>` form, or a rule-specific named trailer (e.g. `Test-Adjusted:`, D-005) — permanent, searchable, machine-checkable. Never chat-only approvals, and never PR labels — with one deliberate exception, the `allow-binaries` label (D-007), whose trailer-ization that record defers, with a named reactivation trigger.

- **PR verification blocks are forced-choice** — each block offers its applicable options (verified-with-evidence / N-A-with-reason / deferred-with-reason-and-follow-up) and exactly one is ticked — so "didn't check" can never hide inside "didn't tick".
- **No silent lanes.** An allowed-to-fail CI job ends with an explicit failure beacon; a file-targeted gate fails loud when its target file is missing (a moved file may never turn a gate green); and a dormant gate lane owns its activation condition — it detects when it becomes needed and fails until configured or explicitly declared off with a reason.
- **Exception lists are ledgers**, governed by four rules: **a reason per entry**, inline; **a ceiling** on size, so growth is a visible diff; **staleness fails loud** — an entry no longer needed fails the gate until removed; **itemized, never blanket** — entries name specific items, never a directory or category. One boundary: honesty- and security-critical findings (a real secret, a fabricated claim) are **never ledgerable**.

The concrete enforcers of this doctrine — bound into `make verify` and the PR gates — are process-layer tooling, mapped in Enforcement.



Left: a convention becoming a rule, a hook, a gate, and finally a merge. Right: decisions superseding one another while the history stays readable

Four layers

Enforcement is arranged in layers, ordered by how early they catch a problem and how hard they are to bypass.

Prose — the agent-guidance file and the docs. Zero enforcement, total coverage. This is the only layer that can express judgement, and it is where anything unenforceable belongs.

Client hooks — scripts that intercept an agent's actions before they happen. Immediate feedback at the point of the mistake, when correction costs nothing. They fail open: a hook that is broken, disabled, or simply not installed protects nothing.

CI gates — workflows that run server-side: most on every pull request, one only on promotion PRs, one on issue events. Slower than a hook, but they survive every client configuration and cannot be skipped by a contributor who never installed the hooks. They fail closed.

Branch protection — repository settings that make the CI gates required rather than advisory, and prevent direct pushes to the long-lived branches outright. The blueprint's own pages count branch protection as part of the server authority, alongside the gates it hardens; their fourth layer is a different thing entirely — the declared-exception channel described below.

One clause in the doctrine — the permission-prompt layer "counts for nothing", being mode-dependent — has more bite than it looks. The blueprint does ship a list of commands refused outright: force-pushes, hard resets, merges, repo and issue deletion; the harness chapter prints it in full. But it is not that list which stops a force-push; the git guard and branch protection do. The list keeps the sharpest edges out of easy reach, and no more. Mistaking it for the thing that protects the repository is precisely the error the doctrine is written to prevent, because sessions legitimately run prompt-free.

The doctrine's most exacting requirement is that each layer **declares its failure direction** — and that when a client-side check defers something, the deferral names which later layer catches it. No rule is allowed to quietly evaporate between layers.

The gates

WORKFLOW FILE	GATE NAME
<code>adr-gates.yml</code>	ADR gates
<code>branch-flow-guard.yml</code>	Branch flow guard
<code>docs.yml</code>	Documentation
<code>issue-close-guard.yml</code>	Issue close guard
<code>issue-link-guard.yml</code>	Issue link guard
<code>repo-hygiene.yml</code>	Repo hygiene
<code>security.yml</code>	Security

Alongside them, the harness ships 7 client-side hooks, wired to four points in a session's life:

FIRES ON	HOOKS
<code>PreToolUse</code>	<code>guard-command-policy.py</code> , <code>guard-git.sh</code> , <code>guard-adr.sh</code> , <code>guard-issue-close.sh</code>
<code>SessionStart</code>	<code>session-start.sh</code>
<code>PreCompact</code>	<code>pre-compact.sh</code>
<code>Stop</code>	<code>stale-working-docs.sh</code>

Of the four wired to intercept tool calls, three deny. The fourth, `guard-command-policy.py`, is the approve-hook, and it exists to solve prompt fatigue rather than misbehaviour: where the git guard asks "must this be blocked?", it asks "can this be approved without asking?" — and

it never blocks, so a gap in it costs a prompt rather than a mistake. The three hooks wired elsewhere in the session's life never block anything: they inject orientation at session start, remind before compaction, and nudge about stale working files at session end.

The security row is the one the table undersells: a merge-blocking secret scan (a version-pinned gitleaks) plus a dependency audit that runs on every pull request and again on a weekly schedule, with Dependabot keeping the actions and docs toolchain current. The audit never blocks a merge — which points at the principle deciding what does. A check may block merges only if its verdict is about the pull request's own change; the audit's verdict tracks external CVE feeds, so making it required would let a third-party disclosure freeze every open PR, including the one bumping the vulnerable pin.

Each guard has a regression suite that runs inside `make verify`, and therefore in CI on every pull request. This is a deliberate consequence of the testing policy: scaffolding is code, so a broken guard fails the build rather than silently permitting whatever it was supposed to prevent.

One verification entryptoint

There is a single command for "is this ready", and it is the same command locally and in CI:

```
make verify
```

What it runs, read from the blueprint's own Makefile:

- `bash scripts/test_guard_git.sh`
- `bash scripts/test_guard_adr.sh`
- `bash scripts/test_branch_flow_guard.sh`
- `bash scripts/test_adr_unlock_decision.sh`
- `bash scripts/test_issue_link_guard.sh`
- `bash scripts/test_release_gate.sh`
- `bash scripts/test_guard_issue_close.sh`
- `bash scripts/test_issue_close_guard.sh`
- `bash scripts/test_guard_command_policy.sh`
- `bash scripts/test_stale_working_docs.sh`
- `python3 scripts/test_guardlib.py`
- `if [ -f scripts/check_bootstrap_complete.sh ]; then bash scripts/check_bootstrap_complete.sh --self-test; fi`
- `for f in scripts/*.sh .claude/hooks/*.sh; do bash -n "$$f" || exit 1; done`
- `mkdocs build --strict`
- `python3 scripts/check_ci_gates.py --self-test`
- `python3 scripts/check_ci_gates.py`
- `python3 scripts/check_docs_truth.py --self-test`
- `python3 scripts/check_docs_truth.py`

The two `check_*.py` entries are the unusual ones. `check_ci_gates.py` is a meta-gate: it pins the gate configuration, so a workflow cannot be silently unwired, and a new deny-hook shipped without a test suite fails the build instead of passing unnoticed. `check_docs_truth.py` is the semantic docs checker described earlier. Both run their own self-tests first, so the checkers themselves stay pinned.

Exceptions leave a record

Rules meet situations their authors did not foresee, so the blueprint provides a way through — but never a silent one.

A deliberate bypass is declared as a **commit trailer**: a machine-readable line naming the rule being skipped and the reason. The exception travels with the commit, permanently, and is visible to anyone reading the history. There is no verbal approval, no quiet skip, and no configuration flag that disables a check without leaving evidence. The blueprint is precise about how far the machinery goes: two trailers are CI-checked today — the ADR unlock and the registry-sync skip — and the rest are convention that pull request review upholds, a gap the doctrine records as a deferred item rather than leaving implied.

Where a rule maintains a list of exceptions, that list is a **ledger**, governed by the four rules the doctrine block above spells out — so it can be audited and pruned rather than growing forever as an accumulation of forgotten special cases.

ADVISORY RULES ARE LABELLED AS SUCH

Some rules cannot be mechanically enforced — knowing when to stop, when to ask, when a simpler approach exists. The doctrine's requirement is not that everything be enforced, but that everything be honest about whether it is. A rule marked advisory says so, and names why enforcement is not possible. What it may not do is imply a guarantee that nothing provides.

The agent harness

Everything in `.claude/` governs how an AI coding session behaves in the repository. It is the layer that turns written conventions into things an agent actually does, and it is the part of the blueprint with the least prior art.

One map, the rest on demand

The harness is built around a budget. Anything loaded at session start is paid for by every session whether the task at hand needs it or not — the orientation hook's own header states the discipline as bluntly as it can be put: "every unconditional line taxes every future session's context, forever." So instead of one large always-loaded rulebook, the guidance is distributed across tiers that load at different moments:

WHAT	WHEN IT LOADS	WORDS
<code>CLAUDE.md</code> — the map	every session, always	2,142
Path-scoped rules	when a matching file is read or edited	701
Skills	when a situation matches a one-line description	2,278
Ritual commands	on invocation	3,371
The docs site	when the routing table names a page for the task	16,545

The always-loaded tier is kept lean: for every word a session carries from the start, roughly 11 wait on disk until something asks for them. (Counted from this template checkout at build time; a seeded project's map is thinner still, since bootstrap deletes the blueprint-only material `CLAUDE.md` carries here.) What earns a place in the map is only what must bind everywhere — the hard rules, the autonomy contract, and the routing table — each entry a pointer into the docs, never a copy of them. Even growth is demand-driven: a hard-won lesson starts in the lessons ledger, graduates to a one-line rule in the map only if it keeps being violated, and becomes an automated check only if it recurs after that. The map grows by promotion, never by default.

The pointer discipline guards itself, economically: the rule that keeps `CLAUDE.md` thin is itself a path-scoped rule, loading only at the moment someone edits `CLAUDE.md` and costing nothing in every other session. The blueprint is candid about the trigger's limits, too — its own notes record that a path-scoped rule does not fire when a new matching file is created, and tell the operator to re-verify the semantics against each harness version rather than trust them.

The remaining channels deliver rules at moments rather than from files. The session-start hook injects orientation — the current branch, a dirty tree left by a previous session, the newest changelog heading, the latest lesson titles — printing only lines that carry signal, and headings rather than bodies, so a session gets its history for a handful of tokens and drills down only if it needs to. A guard's block message carries the rule and its recovery path at the exact moment of violation, which is why a hook-enforced rule does not also need to sit in the window all day. The pull request and issue templates put the closing-keyword gram-

mar and the verification checklist inside the very artifact being composed, at the moment of composing it. Rules are in the window when they are needed because the moments of need each have a delivery channel — not because everything is loaded all the time.

Hard rules and the autonomy contract

The guidance file opens with a short list of hard rules — the things an agent must never do without an explicit request in the current session. Never push to the promoted branch. Never commit directly to a long-lived branch. Never merge your own pull request. Never force-push or rewrite published history. Never delete a branch you did not create. Never commit a binary. Never close an issue. Never edit a Decided decision record.

Each is backed by a client-side hook and a server-side layer — a CI gate or branch protection — and every block message names the recovery path: the rule tells you what to do instead, not merely that you have failed.

The git guard's most agent-shaped check is on pushes. An open pull request's branch is append-only, and before any push the guard reads the PR's state from the platform — because pushing to a branch whose PR has already merged or closed succeeds in git while landing nowhere, the exact failure that produces a confident "change landed" report about a change nobody will ever see.

The **autonomy contract** covers the larger territory where judgement is required, and is explicitly advisory: no hook can enforce judgement, so pull request review and the honest-reporting habit are its backstop.

Proceed without asking covers reversible work on a feature branch — implementation choices inside a decision's boundaries, refactors, tests, documentation fixes, filing issues. Where a task is ambiguous, the agent picks the most likely reading, states the assumption, and proceeds.

Stop and ask first covers actions that are irreversible, that contradict a standing decision, that add a dependency or touch CI or repository settings, that spend money or publish something, or where a wrong guess costs more than about half an hour of rework. Persistence is bounded too: one workaround per problem, and three failed attempts on the same failing check mean stop and escalate, not a fourth attempt.

Two clauses in the contract cut against how agents usually behave.

Instructions are reviewed, not merely executed. An instruction — including one from the operator — is treated as a starting point rather than a verdict. If the docs, the standing decisions, or the agent's own analysis argue against it, the agent is required to say so and lay out the alternatives with a recommendation, then leave the choice to the operator. The contract states plainly that silent compliance with a flawed instruction is as much a failure as silent deviation from a sound one.

A failed question is a hard block, never a default. If a question to the operator fails, is dismissed, or goes unanswered, the agent must not proceed on an assumed answer — not even the option it recommended. It retries; failing that, it stops and waits. This closes a specific and dangerous failure mode in which a dialog error is silently reinterpreted as consent.

What a session may run unasked

The rules above are about judgement. Underneath them is a blunter mechanical layer: a list of commands the harness runs without stopping to ask, and a list it refuses outright.

Pre-approved: `ls`, `git status`, `git diff`, `git log`, `git show`, `git branch`, `git fetch`, `git add`, `git commit`, `git checkout -b`, `make verify`, `mkdocs build`, `pytest`, `ruff check`, `gh issue view`, `gh issue list`, `gh pr view`, `gh pr list`, `gh pr checks`, `gh label list`.

Notice the shape of the list. Reading the repository is free — status, diff, log, show, branch. So is the verification entrypoint, and so is looking at issues and pull requests. Committing is on the list; pushing is not. The line falls between changing your own working copy and changing anything anyone else can see.

Refused outright: `git push --force`, `git push -f`, `git reset --hard`, `gh pr merge`, `mcp_github_merge_pull_request`, `mcp_github_enable_pr_auto_merge`, `gh repo delete`, `gh issue close`, `gh issue delete`.

Those are the irreversible ones — history rewrites, merges, deletions, and closing an issue, which belongs to the operator rather than the agent.

A pre-approval list built from command prefixes is leakier than it looks, though. Prefix rules match the literal head of a string, so `git -C /repo status`, `MSYS_NO_PATHCONV=1 git show`, and `(make verify || true) | tail -50` all miss a rule that plainly covers what they do. The `ap-prove-hook` closes that gap for the git forms: it splits the command line into segments, strips environment assignments and global flags, and judges the actual verb. Its scope is narrow — read-only reconnaissance, which is where the prompt flood actually comes from — and the `make` pipeline keeps prompting on purpose, because `make` runs arbitrary code no parser can prove read-only. Mutating verbs still prompt, and anything the hook cannot parse or classify falls through to a prompt rather than being waved past. The short list of residual approvals it does make — a handful of local, reversible git ref writes — is documented in the hook's own header rather than hidden.

It also states its own trust boundary rather than hiding it: any script already checked into the repository's own tooling directories is auto-approved, on the reasoning that those scripts passed the same review as the guards themselves. A project wanting the stricter posture flips one flag and pays a prompt per run.

Ritual commands

Multi-step conventions are packaged as slash commands — a known-correct procedure invoked by name rather than reconstructed from memory, with step four intact. Invocation is deliberately the operator's act: 9 of the 11 commands are marked so the model cannot fire them on its own initiative — the operator invokes, the expanded steps then direct the agent. The two exceptions are the two an agent should reach for unprompted: `/tick`, the moment a deliverable lands, and `/checkpoint`, as a window fills. The packaging is also where the words live: a command's steps enter the window only on invocation.

COMMAND	WHAT IT DOES
<code>/adr-new</code>	Scaffold a new ADR — page from template, registry row, nav entry, cross-links

COMMAND	WHAT IT DOES
<code>/checkpoint</code>	Write/refresh a mid-task progress checkpoint that survives context loss
<code>/epic-closeout</code>	Close out an epic — note triage, retrospective, pointer comment, verify
<code>/epic-kickoff</code>	Start an epic — issue from template, docs story-page stub, index row
<code>/handoff</code>	Archive <code>.claude/working/</code> contents to <code>.claude/archive/YYYY-MM-DD//</code>
<code>/lock-adr</code>	Revoke a previously issued <code>/unlock-adr</code> token
<code>/note</code>	File a finding as a note issue — cross-linked to its epic, never a sub-issue
<code>/promote</code>	Run a release train — operator-decided bump, caboose PR, promotion PR
<code>/session-close</code>	End a working session — changelog entry, findings sweep, records commit, state check
<code>/tick</code>	Tick delivered deliverable box(es) on an issue — attest each with evidence, then the anchored body edit
<code>/unlock-adr</code>	Unlock a Decided ADR for maintenance edits in this session (1h token)

Skills

Skills are protocols the agent loads when a matching situation arises: `adversarial-verify`, `careful-coding`, `context-engineering`, `honest-numbers`, `reproduce-first`. Only each skill's one-line description is resident; the protocol body stays on disk until it is needed. And one of them, `context-engineering`, is the economy managing itself at run time: it sets a working band for window utilisation, ranks the failure modes — incorrect context outranks missing context, which outranks noise — and sends search-heavy discovery into subagent windows so the noise lands there and only a distilled summary returns.

Three carry particular weight. `adversarial-verify` is the sharp end of the definition of done: a load-bearing claim gets an independent check that tries to refute it, in three grades scaled to risk, proposed with its cost so the operator can dial it up, down, or off. `reproduce-first` requires that a reported failure be reproduced against the unmodified project before any diagnosis or fix — and includes a halt condition: if the reproduction contradicts the brief, the agent stops and reports rather than proceeding on a premise the evidence has just falsified. `honest-numbers` is a pre-publication checklist for any measured figure or capability claim, requiring that numbers name their source and sample size, and that claims about what the project can do state where that capability actually exists.

Working documents and scratch space

Agent working notes live in `.claude/working/`, never in the documentation tree. The separation matters: `docs/` is canonical and reviewed, whereas working notes are provisional thinking that would pollute it. At the end of a task the notes are archived rather than deleted, so the reasoning survives without being mistaken for doctrine.

The same directory is the harness's answer to context loss. When a window fills, the conversation is compacted — summarised in place — and anything not written to disk may not survive. `/checkpoint` writes the task's state to a one-screen file: end goal, approach, steps done

with evidence, next step — drawn from the files and `git log`, never from recollection. A hook fires just before compaction, the one moment loss is knowable in advance, to say: write it now. And on resume, the standing instruction is to trust the checkpoint and the git history over memory. Files survive compaction; chat history may not.

Single-source policy seams

Three small data files exist so that a policy cannot be expressed in two places and disagree with itself.

`.claude/asset-dirs.txt` lists the directories where binary assets are permitted. It is read by both the client-side git guard and the CI hygiene workflow — one datum, two enforcement points, no possibility of drift between them. `.claude/docs-truth.txt` does the same job for the docs truth checker, and `.claude/release.txt` for releases: it names the project's version file and release log for the release gate and the `/promote` ritual alike, and a project that does not cut releases declares that posture in the same file instead of unwiring the gate.

This is the enforcement doctrine applied to its own configuration, and it is the pattern to copy when adding a rule: put the policy in one file, and have every layer read it. It is also the cheapest kind of rule there is: a parameter held as machine-read data occupies no context at all, and surfaces to the agent only inside a block message at the moment of violation.

The lifecycle rituals

A template that only works at the moment of creation is a snapshot. The blueprint treats the relationship between template and project as ongoing, and gives it four named rituals — each a document an agent session executes by name — plus the release train that cuts the versions two of them trade in.



The four rituals as a cycle: the template seeds a project, improvements are harvested back, and existing projects update or adopt in place

Birth — **BOOTSTRAP.md**

Run once, in a fresh repository created from the template. The session interviews the operator for the project's name, its one-line description, its domain areas, its posture toward binary files, which optional modules to apply, and whether to publish the documentation site. It fills the placeholders throughout the tree, writes the sections that require judgement rather than substitution, and then submits to a completion gate.

The gate is what makes the interview binding. `check_bootstrap_complete.sh` fails if any placeholder token survives anywhere, or if any unresolved editorial marker remains. Only once that passes and `make verify` is green does the session delete the template machinery — the ritual documents, the module payloads, the blueprint directory, the bootstrap gate itself — and commit the result.

That deletion is deliberate: leftover scaffolding is exactly the kind of thing that confuses a future contributor into thinking it means something. What survives is a two-line stamp, kept on purpose — a README footer and the birth commit's message, both naming the blueprint version — because the update ritual later reads that stamp to know which release to merge from.

Bootstrap's commit to the default branch is the code repository's single sanctioned direct push, and the git guard permits it only while the repository is demonstrably un-bootstrapped. (The paired data repository, where one exists, is deliberately different — direct pushes from tooling are its normal write mode.) Any inconclusive signal — no remote, a network error, an ambiguous response — fails closed and blocks the push.

Staying current — `UPDATE.md`

The template keeps improving after your project is created. `UPDATE.md` pulls a newer version into an existing project as a three-way merge against the tagged release in the project's blueprint stamp — the version it was seeded from, or last updated to.

The merge is tier-aware because every shipped file declares its tier: process files are literal (byte-identical in every seeded project); config files are tokenised; identity files are generated. Updates never touch the project's identity — its README body, its filled registries, its ADR contents, its history — and every conflict is surfaced to the operator as a judgement call, never auto-resolved.

The critical constraint is that **decisions are re-proposed, never copied**. If the template changed its mind about something, the project is shown the new reasoning and decides for itself. A template does not get to silently overwrite a downstream project's recorded judgement, because the project may have had reasons the template knows nothing about.

Retrofit — `ADOPT.md`

For a mature project that was never seeded from the template and cannot be. There is no common ancestor, so nothing merges; this is judgement work, staged in operator-approved waves, one pull request per wave, budgeted in days rather than hours.

Its ground rules are notably restrained. **Adopt invariants, not shape** — what transfers are the rules, not the layout. The project keeps its own directory structure, branch names, and tooling; the rules are mapped onto what exists. **Graft, never replace** — an existing agent-guidance file stays the base and keeps its voice, with blueprint rules merged into it. The ritual's no-go zones are absolute: never rewrite history, never renumber existing identifiers, no bulk reformatting, and nothing lands without its wave's approval.

It also begins with an inventory that forbids assumption: every claim about the existing project must cite file evidence, and the resulting gap-and-conflict matrix is high-risk enough to warrant independent adversarial verification before anything is built on it.

Feed back — `HARVEST.md`

Improvements discovered while working in real projects flow back into the template.

`HARVEST.md` runs in the blueprint repository and collects candidates two ways: `note` issues in live projects flagged for the blueprint, and diffs of the literal process files — which started byte-identical in every seeded project, so every delta is a candidate by construction. The batch is then judged, not merged wholesale.

The filter is whether an improvement is genuinely project-agnostic. Something that works because of one project's particular domain is not a template improvement; something that would have helped every project is. Accepted changes are versioned and tagged, which is what makes `UPDATE.md` possible downstream.

Releases

Those tags have to be cut somehow, and the mechanism is one every seeded project inherits: promotion from the integration branch to the promoted branch, treated as a release rather than a merge. The `/promote` command runs a two-pull-request train: a caboose that bumps the version and writes the changelog entry, then the promotion itself. The version bump is an explicit stop where the operator decides the number — the agent does not choose it — and a release gate checks the train is properly formed before it can land.

Bootstrap choices

Most of what the blueprint installs is fixed. A handful of things genuinely cannot be — they depend on what you are building — and those are settled once, in an interview at the start of the bootstrap session. This chapter is the catalogue: what you will be asked, what each answer commits you to, and which decisions the ritual deliberately leaves for afterwards.

What the interview asks

ASKED	WHAT IT SETTLES
Identity	project name, one-liner
GitHub coordinates	owner and repo slug (default: <code>parse git remote get-url origin</code>)
Domain areas	which topic sections the docs will grow (architecture? design? content?): used for the CLAUDE.md routing rows, <code>area:*</code> labels, and mkdocs nav pruning
Binary policy	one posture from <code>modules/README.md</code> → Binary policy; if in-repo assets: the sanctioned asset directories
Modules	<code>python-package</code> ? (then: package name, and the architecture seams), <code>data-repo</code> ? (then: artifact kinds ...)
Docs publishing	publish docs to GitHub Pages on each merge to <code>development / main</code> ? (default: no.) For private projects ...

Anything already answered in the message that invoked the session is skipped, so a well-specified request can carry the whole interview.

THE PUBLISHING QUESTION IS THE ONE WITH A TRAP IN IT

Publishing the documentation to GitHub Pages defaults to **off**, and for a private project it should stay off. A Pages site can be publicly reachable even when the repository is not, so switching it on for a private project publishes the project's internal documentation to the open web — and does so again on every subsequent merge. Turn it on only when the docs are meant to be public.

The binary posture

This is the interview's most consequential answer, because it is the only one that rewrites a decision record rather than filling in a blank.

Strict / split (default) — No binaries in the code repo, ever; durable run artifacts live in the paired data repo. Apply the `data-repo` module; leave the hook/CI allowlists empty. Right for research platforms, experiment-producing projects, anything long-lived

In-repo assets — The project's binaries are part of the product (website images, game assets, design files). Name the sanctioned asset directories (e.g. `assets/`, `static/`) and list them, one per line, in `.claude/asset-dirs.txt` ...

Keep the default when unsure — For small projects with no clear binary story, posture 1 without the data-repo module (strict rule, no split) is the cheapest safe choice — the `allow-binaries` PR label remains the explicit per-case exception ...

The guard and the CI lane enforcing binary hygiene are live the whole time — the **PROPOSED** status marks the pending posture choice, never a dormant guard. The chosen posture is wired into three places in the same commit: the hard-rule bullet in the agent-guidance file, the `.claude/asset-dirs.txt` data file that both the guard hook and the CI gate read, and `D-007` itself. Promoting it to **DECIDED** goes through the same unlock the lock demands of every other decision record; the birth commit carries the corresponding trailer.

Two rules hold regardless of posture. Where LFS is part of the answer, its patterns are wired before any matching binary exists — LFS added after the fact still leaves the old blobs in history. And the hygiene gate keeps a second lane no waiver lifts: an oversized file outside LFS fails even under the deliberate binaries exception, so the binary waiver can never smuggle a huge text file.

If none of the postures obviously fits, take the strict default: reversing it later is an ordinary superseding record rather than a migration.

Modules

Modules are optional payloads applied during bootstrap and deleted with the rest of the machinery afterwards. Declining one costs nothing; its payload simply dies with the `modules/` directory.

MODULE	PURPOSE
<code>data-repo</code>	The paired data repository — durable home for run artifacts (metrics, plots, videos, checkpoints) ...
<code>lfs-assets</code>	Git LFS for authored binary assets (3D models, textures, audio ...
<code>python-package</code>	The <code>python/</code> src-layout skeleton: one installable package whose subpackages mirror the architecture seams ...

What is not a choice

Two things people expect to be asked about, and are not.

The branch model. It is fixed by `D-003` — feature branch, pull request into the integration branch, then a promotion pull request into `main`. You do not pick it at bootstrap. What does happen at setup is that the branch topology gets created, because a GitHub template copies files and nothing else: no branches, no labels, no protection rules, no Pages configuration, no repository variables. That gap is why the template ships a setup script.

Running it creates the integration branch and makes it the default, applies branch protection with the shipped gates as required status checks, creates the label taxonomy including any domain-area labels from the interview, and sets the merge behaviour. Pages publishing is added only when explicitly opted into.

The process itself. The decision registry, the enforcement doctrine, the testing policy, the honesty rules — these arrive settled; the next chapter inventories what that means. What you extend is the set: your project adds `D-008` onward as its own decisions accumulate.

What bootstrap defers to you

Not everything can be finished inside the session. After the completion gate passes and before the machinery is deleted, the ritual writes a `POST_INIT_CHECKLIST.md` naming what is left, so nothing quietly goes missing:

- **The GitHub setup run**, whenever the GitHub CLI was unavailable in the session — written out as the exact command, at the top of the list, because until it runs none of the server-side layer exists: no integration branch, no protection, no labels.
- **A licence.** The template's own licence file is deleted during bootstrap. Its terms explicitly permit this — the blueprint is MIT with a template-use waiver, so a seeded project inherits no obligation and starts licence-free. That also means it starts with no licence until you add one, which for a public repository is a meaningful state. The checklist asks for the project's choice, or for the deferral to be recorded with a written trigger for revisiting it.
- **Large-file storage.** If the LFS module was applied, `git lfs install` still has to be run once per machine by each person who clones. The patterns ship in the repository; the local hook does not.
- **The paired data repository**, if that module was chosen and the GitHub CLI was unavailable in the session — including pushing the seed the module staged, which is destroyed if it is not rescued before the machinery is deleted.
- **Secrets and integrations**, which no template can carry.
- **Turning off the template flag**, so the new repository does not itself advertise as a template.

The pattern across all of them is the same: bootstrap does what it can verify, and writes down what it cannot, rather than reporting a clean finish over an unfinished job.

What a project inherits

This chapter is the concrete answer to "what do I actually get". It describes the state of a repository the moment bootstrap finishes and the machinery deletes itself.

Filled, versus seeded empty

The distinction that matters most, and the one least obvious from a file listing, is that the blueprint ships two different kinds of content.

The machinery arrives complete. The process pages, the enforcement doctrine, the guard hooks and their regression suites, the CI workflows, the ritual commands, the skills, the agent permission lists, the issue templates and label taxonomy — all of these are finished, working, and identical in every project (the interview's domain-area labels aside). They are not examples to adapt. They run on day one.

The registries arrive empty — almost. The requirements registry, the open questions, the roadmap, the lessons file, the epic index: structures with no content, waiting for the project to have some. Bootstrap seeds the identity — the name and one-liner, a written vision page — and the changelog's first entry, recording the birth itself; every entry after that is the project's own to write.

This split is the design. Copying another project's requirements would be worse than useless, but copying a working enforcement doctrine is exactly what you want. The blueprint is opinionated about how to record things and deliberately silent about what you will record.

Aside from the identity — name, one-liner, GitHub coordinates — what varies between two seeded projects is only what the previous chapter settled: the binary posture, the modules applied, the domain areas, and whether the docs publish. Everything else is identical.

The inventory

AREA	WHAT LANDS
docs/	MkDocs Material site: 16 process pages, 7 seeded decision records with their registry, empty direction and records registries, 7 reusable page templates
.github/	7 gate workflows, label taxonomy, issue forms, tri-state pull request template
.claude/	7 hooks wired to the session lifecycle, the command allow- and deny-lists, 11 ritual commands, 5 skills, path-scoped rules, agent scratch space, single-source policy seams
scripts/	22 scripts (one, the bootstrap completion gate, deletes itself at bootstrap): GitHub setup, label sync, the CI meta-gate, the docs truth checker, and the guard regression suites
Root	CLAUDE.md agent guidance, Makefile with the verify entrypoint, editor and git configuration

Work tracking

Documentation holds truth; issues track work. The two are kept firmly apart, and the seam between them is one of the most heavily gated parts of the system: a pull request fails CI if its text would keyword-close an epic that still has open sub-issues, or any issue with unchecked deliverable boxes — and the docs truth checker fails documentation that cites a closed issue as open. What stays convention is only placement: what counts as a sub-issue, what becomes a note.

Epics group native sub-issues, which are build tasks. A finding that is not a build task — an observation, a concern, something noticed in passing — becomes a `note` issue and is explicitly never a sub-issue, so the epic's completion criteria stay meaningful rather than accumulating everything anyone ever noticed. Notes are triaged when the epic closes.

An issue closes at the moment its deliverable is met rather than being batched to the end of a work session; its deliverable checkboxes are ticked by the session that delivered them, with evidence, and outcomes are written back — a run or build result lands as a readout comment on the issue that produced it. Closing authority belongs to the operator: an agent may not close or delete an issue. Enforcing that has a twist, because an agent acts with the operator's own credentials and the platform cannot tell the two apart. The hook blocks the attempt at the client; the server side uses the one signal GitHub does record — whether a close was performed via an app — and reverts app-mediated closes after the fact, reopening the issue with an explanatory comment, while a human's click stands.

Definition of done

The bar is written down, and it is not "the tests pass".

The testing policy beneath it is deliberate in what it refuses: no coverage thresholds, because a percentage target trains authors to write tests for the metric. Test-writing is event-driven — a bugfix names its regression test — and weakening or deleting a test is a declared exception carrying a `Test-Adjusted` trailer, never a silent loosening.

Done means `make verify` passes — actually run, with failures fixed in the order they surface — **and**, for a behaviour change, that the change itself was exercised and the output looked at. The verification commands and their results go in the summary. Anything that could not be run in the environment is stated explicitly, never implied green.

That last clause is the whole point. The system's tolerance for an honest "I could not verify this" is high; its tolerance for an unverified claim presented as verified is zero.

Colophon

This report describes **Project Claude Blueprint v1.2.0**, commit `b2df960` dated 2026-08-14. It was generated on 2026-08-25.

How it is produced

The report is compiled by a separate repository, `CLAUDE_blueprint_report`, which exists so that the blueprint itself carries no report-building scaffolding. The blueprint stays clean; the machinery lives next door.

The document is a **hybrid**. Its prose spine is written by hand — the narrative, the framing, the judgments about what matters. Its facts are pulled out of a blueprint checkout at build time: the decision registry, the gate list, the command table, the module inventory, the contents of `make verify`, and every count quoted in the text. Nothing factual in these pages is typed twice.

The pipeline is markdown → directive expansion → HTML → PDF, rendered with WeasyPrint against a CSS Paged Media stylesheet.

How it is kept honest

Three mechanisms guard against the failure this report would otherwise be most prone to — describing a version of the blueprint that no longer exists.

Derived facts. When the blueprint gains an eighth decision record, this report says eight the next time it is built, without anyone remembering to update a number.

A placeholder guard. The blueprint is a template, so parts of it contain unfilled tokens and editorial markers meant for the bootstrap session. The build fails if any of them reach the assembled document, rather than printing a literal placeholder in front of a reader.

A provenance manifest. Every build records which blueprint files it read and a digest of each. Running the build with `--drift` compares those digests against the current blueprint and reports exactly which cited sources have moved since the report was last cut — so maintaining this document means reviewing a short list of changed files rather than re-reading the blueprint.

Scope and limits

This is a report about process, written for a human reader who wants to understand what the blueprint does before adopting it. It is not the documentation. The blueprint's own `docs/` site remains canonical, and where this report and that site disagree, the site is right and this report is stale.

It also describes the template, not any project seeded from it. A live project will have filled the empty registries with its own content, may have chosen different modules and a different binary posture, and will have accumulated its own decisions beyond the 7 that the template ships.

Provenance

Compiled from 7 workflow definitions, 11 command definitions, 7 hook scripts, 3 module manifests, the decisions registry indexing its 7 records, the bootstrap and module manuals, and the blueprint's Makefile, harness settings, and cited docs pages. The exact file list and digests are recorded in `out/provenance.json` alongside this document.